
The Plateau Knows: Pre-Registered Evidence for Early Predictors of RL Training Outcomes

Anonymous authors
Paper under double-blind review

Abstract

Grokking – a long plateau at chance performance followed by an abrupt jump to high accuracy – was first characterized in supervised learning, where several signals (effective rank of hidden representations, weight-norm and loss-curvature dynamics) are known to precede the transition. A recent line of work shows that an analogous plateau-then-breakthrough pattern also occurs during reinforcement learning with verifiable rewards (RLVR), driven by reward schedule rather than by epoch count. Whether the supervised-learning precursor signals transfer to this qualitatively different training regime – a sparse, non-differentiable reward rather than a dense gradient – has not been tested. We build a fully reproducible, compute-constrained testbed (a $\sim 0.8\text{M}$ -parameter transformer, single T4 GPU) to test this directly, and along the way discover and fix a training-stability failure mode in group-relative policy optimization (GRPO) that prevented breakthroughs from occurring at all under the standard per-group standard-deviation-normalized advantage estimator at small group sizes; removing the normalization term restores learning. We then run a three-round, pre-registered study of four candidate early-warning signals. The first pre-registered signal (reward variance) fails to replicate under a properly powered follow-up and we report that failure directly rather than substituting a better-looking result. A second signal, effective rank, is pre-registered on a disjoint, held-out batch of seeds and predicts eventual training reward with a correlation that independently replicates in both seed batches ($r = 0.647$, $p < 0.001$ pooled; $r = 0.663$ on the held-out batch alone) and survives controlling for the model’s current reward level ($r = 0.630$, $p < 0.001$). The same signal’s ability to predict the discrete, threshold-based breakthrough event, however, does *not* independently replicate (AUROC 0.730, $p = 0.039$ pooled, but 0.277 on the held-out batch alone) – a result we take at face value rather than explain away. We report this distinction as the paper’s central finding: an early representational signal predicts *how well* RL training eventually goes, but not reliably *whether* it crosses a specific discrete threshold, at least at the scale and sample size we could test. We also find that breakthrough outcomes cluster strongly by random seed independent of reward schedule, and that a second, unregistered signal (policy entropy) shows an even larger effect that we explicitly flag as exploratory rather than confirmed. All code, logs, and per-run trajectories are released for reanalysis.

1 Introduction

A model trained on a small algorithmic dataset can sit at chance-level validation accuracy for thousands of steps, apparently having learned nothing, and then generalize almost overnight. Power et al. [1] named this *grokking* and showed it reliably in supervised settings; a substantial body of follow-up work has since identified signals that precede the jump – representation geometry [2, 3],

circuit formation [4], optimizer dynamics [5, 6], and weight or activation norms [7]. All of this work shares one structural property: the training signal is a dense, differentiable cross-entropy gradient, computed at every token, every step.

Sun et al. [8] recently documented a strikingly similar plateau-then-breakthrough pattern in a categorically different setting: reinforcement learning with verifiable rewards (RLVR), where the training signal is sparse, often binary, and non-differentiable. Their central empirical claim is that the reward *schedule* – specifically, whether training begins with a dense, graded reward before switching to a strict pass/fail signal, or uses the strict signal from the start – determines whether a model breaks through at all within a fixed compute budget. This raises a natural question that, to our knowledge, has not been directly tested: do the early-warning signals identified for *supervised*-learning grokking transfer to this RL setting, where the underlying optimization mechanism is fundamentally different?

Answering this required building a testbed from scratch under a real compute constraint (a single Kaggle T4 GPU, an academic-scale budget by design, not a simplification chosen for convenience), and along the way we encountered and had to resolve a problem that turned out to be a finding in its own right. Group relative policy optimization [GRPO; 9], the algorithm underlying the RLVR setting we study, computes its advantage estimate by normalizing each group’s rewards by that group’s own standard deviation. At the group sizes common in practice (we use 16), and especially at reward levels common during the near-chance plateau we specifically want to study, this denominator is frequently small and estimated from very few samples. We show directly that this makes training fail completely at our scale: a model that learns cleanly under supervised cross-entropy on an identical task fails to improve at all under standard GRPO, and removing the standard-deviation normalization term – keeping only the group-mean baseline, in the spirit of REINFORCE [10] – restores learning. Liu et al. [11] independently identify a related normalization bias in GRPO (primarily a response-length bias) and also remove the standard-deviation term as part of their fix; our finding is a different, complementary failure mode of the same normalization choice, isolated at small group size rather than at long response length, and demonstrated by a complete-versus-none learning outcome rather than an efficiency loss.

With a training pipeline that actually produces breakthroughs, we then run the scientific study the paper is built around: a three-round, pre-registered test of whether four candidate signals – policy entropy, effective rank of the policy’s hidden states [12], KL-divergence from a reference policy, and within-group reward variance – measured only during the early, near-chance plateau, discriminate runs that eventually break through from runs that stall. We treat pre-registration as a load-bearing part of the method, not a formality: our first pre-registered signal (reward variance, chosen from an 8-seed pilot) fails to replicate under a properly powered follow-up, and we report that failure in the paper rather than quietly moving on to whichever signal looks best in hindsight. Our second pre-registered signal, effective rank, is registered on a fresh, disjoint batch of 24 seeds never used to select it, and predicts eventual training reward robustly – the correlation replicates independently in both the original and the held-out seed batch, and survives controlling for the model’s current reward level. Under the discrete, threshold-based breakthrough criterion that motivated the whole question, however, the same signal’s predictive power does not survive the same held-out test. We report both results plainly and let the second one narrow the paper’s claim, rather than treating it as a nuisance to write around.

Our contributions are: (1) we identify and fix a training-stability failure mode in small-group GRPO – the standard per-group standard-deviation normalization prevents any learning at our scale, and a plain mean-baseline advantage restores it, verified against supervised learning on an identical architecture and task; (2) we run a three-round, pre-registered study transferring supervised-grokking precursor signals to RL, including a first-round pre-registered failure to replicate, reported honestly, and a second-round signal (effective rank) whose prediction of eventual RL training reward independently replicates across two disjoint seed batches and survives controlling for current reward level; (3) we show the same signal’s ability to predict the field’s motivating question – discrete breakthrough versus permanent stall – does *not* survive the same replication test, and argue this distinction matters for how such signals should be used in practice; (4) we report that breakthrough outcomes cluster strongly by random seed, largely independent of reward schedule; and (5) we release the complete pipeline – 144 runs across two task families, full per-step logs, and every

statistical test we ran, including those that did not confirm our hypotheses – for reanalysis.

2 Related Work

Grokking and its precursors. Power et al. [1] first demonstrated reliable delayed generalization on small algorithmic datasets. Nanda et al. [3] identify *progress measures* – quantities that change smoothly during the apparent plateau even though validation accuracy does not – via mechanistic interpretability of modular arithmetic transformers, the same task family we use. Liu et al. [2] and Liu et al. [7] connect grokking to representation and weight geometry, including a role for effective dimensionality that directly motivates our use of effective rank as a candidate signal. Varma et al. [4] explain the transition via competing memorizing and generalizing circuits, and Thilak et al. [5], Kumar et al. [6] tie its timing to optimizer dynamics, including weight decay – relevant given weight decay’s role in our own training-stability finding (§3.2.1). All of these studies supervised, dense-gradient training; none of it, to our knowledge, asks whether the identified precursor signals transfer to a sparse, non-differentiable RL setting.

RL-induced grokking and RLVR training. RL with verifiable rewards descends from the broader use of RL to align pretrained language models with a feedback signal [13], replacing learned human-preference reward models with automatically checkable correctness, and is now a standard post-training recipe for LLM reasoning [9, 14], typically built on group relative policy optimization [GRPO; 9], a variant of PPO [15] that replaces a learned value baseline with a per-group empirical one. Sun et al. [8] document a grokking-like plateau-then-breakthrough pattern under RLVR training and show that a staged reward schedule – dense feedback before a switch to strict pass/fail – is what separates eventual breakthrough from permanent stall on hard problems; our two reward schedules (§3) instantiate exactly this contrast. Separately, Liu et al. [11] identify an optimization bias in GRPO’s advantage normalization (primarily a response-length bias) and propose Dr. GRPO, removing both the length normalization and the per-group standard-deviation term. Our finding (§3.2.1) is narrower in scope but more severe in consequence at our regime: at small group sizes, the standard-deviation term alone is enough to prevent any learning, not merely bias it, which we verify by direct comparison against supervised learning on an identical architecture and task.

Cheap, training-free diagnostic signals. A separate literature asks whether inexpensive signals computed from a model’s internal state can predict externally meaningful properties without additional supervision. Wang et al. [16] use the effective rank of hidden states across layers and generations to detect hallucination in LLMs without auxiliary modules, which is one of our motivations for treating effective rank as a promising candidate rather than an arbitrary one. Our study differs from this literature in asking about a training-time, not inference-time, prediction target, and in explicitly pre-registering and replicating the test rather than reporting a single-pass correlation.

3 Method

3.1 Tasks

We define two synthetic algorithmic task families over a fixed, finite, seeded space of (a, b) pairs with $a, b \in \{0, \dots, 96\}$ ($97 \times 97 = 9,409$ pairs total, split 50/50 into train and test subsets by a fixed task-level seed shared across every run), matching the classic grokking setup of Power et al. [1] on purpose. TASK-A is single-token: $y = (a \cdot b) \bmod 97$, so a random policy succeeds with probability $1/97 \approx 1.03\%$. TASK-B is two-token and structurally distinct rather than merely longer: $d_1 = (a + b) \bmod 97$ and $d_2 = (a \cdot b) \bmod 23$, requiring two dependent correct predictions, so a random policy succeeds with probability $(1/97) \cdot (1/23) \approx 0.045\%$. TASK-A is our primary task; TASK-B is a cross-task-family robustness check.

An earlier version of TASK-A used three free inputs (a, b, c) , which expands the input space to $97^3 \approx 913\text{K}$ pairs – large enough that a realistic training budget samples almost no pair more than once. We diagnosed this directly: training was flat at the random baseline in both dense and

binary reward phases regardless of step count, and a supervised (non-RL) control on the same three-input task showed identical flatness, ruling out an RL-specific explanation. Grokking depends on a model first memorizing, then generalizing over, a *fixed, repeatedly-seen* training set; an effectively unbounded input space cannot support that mechanism regardless of the optimization method. The two-input, fixed-split design above resolves this by construction.

3.2 Model and Training

The policy is a small causal decoder-only transformer [17] ($d_{\text{model}}=128$, 4 layers, 4 heads, $d_{\text{ff}}=512$; 821,504 parameters total), trained with AdamW [18]. Because a randomly initialized policy has to bootstrap an RL signal purely from what it happens to sample – a categorically harder exploration problem than fine-tuning an already-capable pretrained model, which is what real RLVR pipelines always do – we precede RL with a brief, deliberately incomplete supervised (teacher-forced cross-entropy) warmup, applied identically to both reward schedules so the schedule comparison is not confounded by starting-point quality. This closes part of the gap between our toy setting and what "RLVR fine-tuning" means in practice, while stopping well short of solving the task outright, leaving real work for the RL phase.

3.2.1 The GRPO Advantage Estimator, and Why We Modify It

For each of G rollouts sampled per prompt, standard GRPO [9] computes the advantage of rollout i as

$$A_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G) + \varepsilon}. \quad (1)$$

With $G=16$ and reward levels typical of the near-chance plateau we specifically want to study, $\text{std}(r_1, \dots, r_G)$ is estimated from very few samples and is itself high-variance: a group that happens to land a small denominator by chance receives a disproportionately amplified advantage, while a typical group’s signal is comparatively suppressed. We tested this directly rather than assuming it: on a trivially easy control task (a reduced-modulus version of TASK-A), supervised cross-entropy training on the identical architecture converges cleanly within a few hundred steps, while RL training under Equation 1 shows no improvement over several hundred steps at the same architecture and optimizer settings. Replacing Equation 1 with a plain group-mean baseline,

$$A_i = r_i - \text{mean}(r_1, \dots, r_G), \quad (2)$$

in the spirit of REINFORCE with a baseline [10], restores learning on the same control task under otherwise identical settings. We adopt Equation 2 throughout the paper. We are careful not to overstate this: we do not claim standard GRPO fails in general, only that at our group size and reward regime the standard-deviation term is unstable enough to prevent learning outright, and that the simplest fix resolves it. Liu et al. [11] independently remove the same term as part of a different fix (primarily targeting a response-length bias in long-form generation); our finding isolates a distinct consequence of the same design choice, specific to small group size, and is demonstrated as a complete learning failure on a controlled task rather than an efficiency argument.

We separately found, while diagnosing why an early version of this trainer showed no learning even after removing Equation 1’s normalization, that omitting an entropy bonus lets sampling entropy collapse early: as entropy falls, more groups sample an identical token on every rollout, which zeroes that group’s own reward variance and therefore its gradient contribution – removing the only pressure that could correct the collapse, a self-reinforcing spiral toward an arbitrary, non-improving behavior. We add a standard small entropy bonus (coefficient 0.05) to the loss to break this spiral. We flag one direct consequence for our own study: this means POLICY_ENTROPY (§3.3) is no longer an entirely free, unregularized quantity, which we return to in §6.1.

Two reward schedules are used, taken directly from the mechanism Sun et al. [8] document as separating breakthrough from stalled runs: *two_phase* uses a dense, distance-based partial-credit reward for the first 30% of training, then switches to strict binary reward; *binary_only* uses the strict binary reward from step 0.

3.3 Candidate Signals

All four signals are measured on a fixed evaluation probe, sampled once at run start and held fixed for the run’s lifetime, so trajectories are comparable across steps and are not confounded by which training prompts were drawn that step.

POLICY_ENTROPY: mean token-level entropy of the policy’s output distribution on the probe.

EFFECTIVE_RANK: the effective rank of the policy’s hidden-state representations on the probe, following Roy and Vetterli [12]: given a matrix H of centered hidden-state vectors, let $\sigma_1, \dots, \sigma_k$ be its singular values and $p_j = \sigma_j / \sum_j \sigma_j$ the normalized spectrum. Effective rank is

$$\text{erank}(H) = \exp \left(- \sum_j p_j \log p_j \right), \quad (3)$$

i.e., the exponential of the Shannon entropy of the normalized singular-value spectrum, bounded in $[1, \min(n, d)]$ for an $n \times d$ matrix.

KL_FROM_REFERENCE: KL-divergence between the current policy and a frozen snapshot of the policy at initialization, evaluated at matched positions on the probe.

REWARD_VARIANCE: within-group reward variance, logged directly from each training step’s on-policy rollouts (not the fixed probe, since this is a property of the training batch itself).

3.4 Statistical Methodology

A run is labeled a *breakthrough* if binary success rate stays at or above 0.9 for a sustained window of 100 logged steps; otherwise it is labeled *stalled*. For each candidate signal and each of three early-window fractions (10%, 20%, 30% of total training steps), we compute:

Discrimination AUROC: the area under the ROC curve for using the signal’s mean value during the early window to predict breakthrough versus stall [19], with a bootstrap 95% confidence interval (2000 resamples) and a permutation-test p -value against the null of no discrimination.

Continuous-outcome correlation: the Spearman rank correlation [20] between the signal’s early-window value and the run’s *final* reward (mean over the last 20 logged steps), computed over *every* run regardless of whether it crossed the breakthrough threshold – a higher-power complement to the AUROC test, which discards graded information for every run that improved substantially without quite crossing the threshold. Confidence intervals and p -values follow the same bootstrap and permutation procedure.

Partial correlation: to test whether a signal predicts outcomes beyond simply reflecting the run’s current reward level, we regress the signal’s early-window value on the raw-reward baseline at the same window (ordinary least squares) and re-run both tests above on the residual.

Two baselines are reported alongside every candidate signal: raw reward at the same early window, and elapsed training steps (a signal-free control that should show no discrimination by construction).

Where multiple signal/window comparisons are reported together as secondary or exploratory, we additionally report Bonferroni-corrected p -values [21] across the full family of comparisons in that table. Bootstrap confidence intervals and standard errors follow Efron [22], Efron and Tibshirani [23].

3.4.1 Pre-registration Protocol

We ran this study in three rounds and report the full history rather than only the final round.

Round 1 (8 seeds, pilot): all four signals were computed post hoc; REWARD_VARIANCE looked most promising (AUROC 0.82–0.85 across windows). **Round 2** (24 seeds): we pre-registered REWARD_VARIANCE at window-fraction 0.3 as the confirmatory primary test, based on Round 1. It did not replicate under adequate power (AUROC 0.627, 95% CI [0.312, 0.912], $p=0.32$) – we report this failure directly in §5, not as a footnote. Round 2 also surfaced EFFECTIVE_RANK and POLICY_ENTROPY as strong, unregistered candidates. **Round 3** (24 additional, disjoint seeds, 48 total) pre-registers EFFECTIVE_RANK at window-fraction 0.1 – the Round 2 candidate with the tightest

Table 1: Primary pre-registered result: EFFECTIVE_RANK at window-fraction 0.1, tested on the full 96-run sample. “Given raw reward” rows are residualized against raw reward at the same window (§3.4).

Test	Statistic	95% CI	p
Discrimination (breakthrough vs. stall)	AUROC = 0.730	[0.508, 0.890]	0.0385
Continuous outcome (final reward)	$r = 0.647$	[0.489, 0.756]	< 0.001
Discrimination, given raw reward	AUROC = 0.695	[0.506, 0.852]	0.0870
Continuous outcome, given raw reward	$r = 0.630$	[0.478, 0.751]	< 0.001

confidence interval – as a genuine confirmatory test on data that played no role in selecting it. We flag one honesty caveat throughout: choosing EFFECTIVE_RANK was informed by Round 2’s data, so this is pre-registration relative to the seeds that decide it (24–47), not a blind first guess before ever seeing the task. POLICY_ENTROPY remains unregistered and is reported only as exploratory evidence.

4 Experimental Setup

All experiments run on a single NVIDIA T4 GPU (Kaggle), within an 11-hour session budget, with full checkpoint-level resumability. Per-step throughput is measured empirically at the start of each session (two-point timing within a short calibration run, to separate fixed overhead from marginal per-step cost) and used to auto-calibrate the step budget for the configured seed count and time budget; TASK-A runs used 4,092 steps and TASK-B runs used 6,138 steps at the calibrated rates of 38.8 and 48.0 ms/step respectively. TASK-A uses 48 seeds $\times$ 2 schedules (96 runs); TASK-B uses 24 seeds $\times$ 2 schedules (48 runs), intentionally not extended to match TASK-A in Round 3 since it had already returned a well-powered null result (§5.7) that additional seeds would not inform. Supervised warmup uses 3,000 steps (early-stopped at 60% train accuracy if reached sooner), batch size 128, learning rate 3×10^{-3} . GRPO uses 64 prompts per step, group size 16, sampling temperature 0.9, learning rate 2×10^{-3} , weight decay 0.05, gradient clipping at global norm 1.0. Total measured wall-clock time across the full pipeline (both task families, all analysis, all figures) was 4.36 hours. Full hyperparameters are listed in Appendix A.

5 Results

5.1 Main Sweep

Of 96 TASK-A runs, 7 broke through and 89 stalled by the criterion in §3.4. *binary_only* runs are not simply stalled in an absolute sense: by the end of training, mean reward under *binary_only* exceeds mean reward under *two_phase*, which shows the expected sharp drop at the dense-to-binary switch followed by gradual recovery (Figure 3, Appendix D). This complicates a purely binary breakthrough/stall reading of the schedule comparison and is consistent with §5.6’s finding that outcomes are driven substantially by seed identity rather than schedule alone.

5.2 Primary Pre-Registered Result: EFFECTIVE_RANK at Window 0.1

Table 1 reports the pre-registered primary test alongside its continuous-outcome and reward-controlled counterparts. The binary discrimination test clears conventional significance ($p=0.0385$), though its confidence interval’s lower bound (0.508) sits barely above chance. The continuous-outcome test is markedly stronger: EFFECTIVE_RANK at the early plateau correlates with a run’s eventual final reward across all 96 runs, not only the 7 that crossed the breakthrough threshold. Controlling for the run’s current raw reward at the same window, the correlation is essentially unchanged ($0.647 \rightarrow 0.630$) and remains highly significant, indicating EFFECTIVE_RANK carries information beyond simply reflecting how well the run happens to be doing right now. Figure 1 visualizes both the raw and reward-controlled relationships directly.

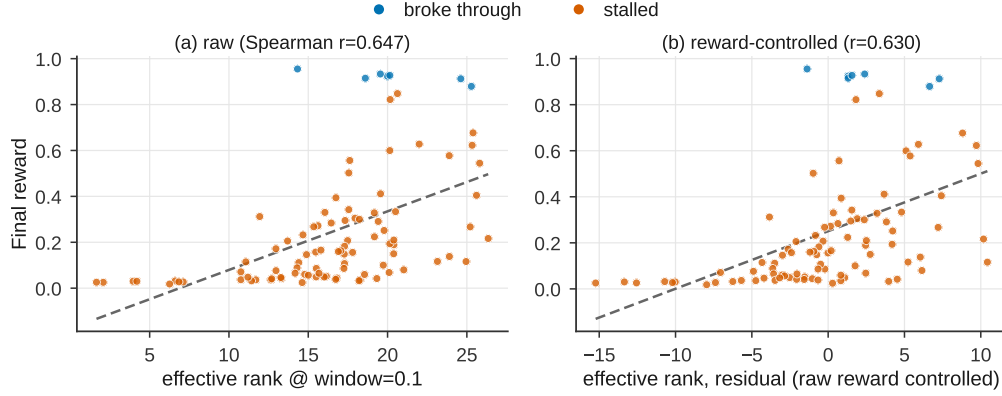


Figure 1: EFFECTIVE_RANK at window 0.1 against final reward, raw (left) and after residualizing out raw reward at the same window (right), colored by eventual breakthrough outcome. The relationship survives residualization essentially unchanged.

Table 2: EFFECTIVE_RANK at window 0.1, split by seed batch. Seeds 0–23 are the same data that originally suggested EFFECTIVE_RANK as a candidate in Round 2 and are not independent evidence; seeds 24–47 are the true, pre-registered confirmatory batch.

Seed batch	n (breakthroughs)	AUROC (p)	r (p)
0–23 (selection batch)	48 (6)	0.790 (0.018)	0.615 (< 0.001)
24–47 (confirmatory batch)	48 (1)	0.277 (0.574)	0.663 (< 0.001)
Both combined	96 (7)	0.730 (0.039)	0.647 (< 0.001)

5.3 The Held-Out Replication Check: Why We Narrow Our Claim

Table 1’s pooled result mixes 48 seeds that were part of selecting EFFECTIVE_RANK as a candidate (Round 2, seeds 0–23) with 48 seeds that were not (Round 3, seeds 24–47). We report both halves separately in Table 2, since only the second half is a genuine out-of-sample test.

The result is unambiguous and we report it without softening: on the confirmatory batch alone, the discrimination AUROC point estimate is *below* chance (0.277) and far from significant. The pooled significance in Table 1 is substantially carried by the non-independent selection batch. Discrete breakthrough discrimination for EFFECTIVE_RANK does not survive an honest, out-of-sample replication test at this sample size – with only one breakthrough in the confirmatory batch, this is also a direct consequence of how rare the discrete event is, not necessarily evidence that no relationship exists.

The continuous-outcome correlation tells a different story: it replicates independently and closely in both batches ($r=0.615$ and $r=0.663$), each individually significant at $p < 0.001$. We take this as the paper’s central, literal finding: EFFECTIVE_RANK measured during the early RL plateau predicts a run’s eventual training reward, robustly and beyond what current reward already indicates, but does not reliably predict whether that run crosses a specific discrete performance threshold, at least at the sample size we were able to collect. These are related but distinct claims, and only the first is independently confirmed.

5.4 An Unregistered but Consistent Second Signal: POLICY_ENTROPY

POLICY_ENTROPY was never pre-registered and its evidential status is correspondingly weaker, but its continuous-outcome effect is large and consistent: pooled $r = -0.817$ ($p < 0.001$; low entropy predicts high eventual reward), and on the Round 3 confirmatory batch alone, $r = -0.868$ ($p < 0.001$) – if anything larger than on the pooled sample. Controlling for raw reward leaves this essentially unchanged ($r = -0.793$). We report this as promising, exploratory evidence, explic-

Table 3: Breakthrough outcome by seed, all 48 TASK-A seeds.

Outcome	Seeds	%
Broke through under neither schedule	44	91.7%
Broke through under exactly one schedule	1	2.1%
Broke through under both schedules	3	6.2%

itly not as a second confirmed result, since unlike `EFFECTIVE_RANK` it was never tested on data collected after it was selected. Full discrimination results for all four signals and both baselines, at every window, are in Appendix B (Table 5); we highlight one entry here because it directly motivates §5.2’s reward-controlled test: the raw-reward baseline itself reaches AUROC 0.868 at window 0.3, exceeding every candidate signal at that window. Without controlling for it, a candidate signal’s discrimination could be substantially explained by nothing more than current reward level, particularly at later windows.

5.5 Combining Signals

A logistic-regression combination of all four signals (leave-one-out evaluated) does not exceed `EFFECTIVE_RANK` alone at any window (AUROC 0.700, 0.700, 0.761 across windows 0.1–0.3, versus `EFFECTIVE_RANK` alone’s 0.730, 0.697, 0.685) – with 96 runs and four features, this reads as the two weak signals (`KL_FROM_REFERENCE`, and to a lesser extent `REWARD_VARIANCE`) diluting rather than complementing the two strong ones. A combination of only `EFFECTIVE_RANK` and `POLICY_ENTROPY`, in contrast, exceeds either alone at every window (AUROC 0.777, 0.775, 0.777), suggesting these two carry genuinely complementary information rather than redundant copies of the same underlying signal.

5.6 Seed-Level Clustering

All 7 breakthroughs across 96 runs come from just 4 of 48 seeds, and 3 of those 4 seeds break through under *both* reward schedules (Table 3). This is a well-powered descriptive fact ($n = 48$ seeds) rather than a new statistical test, and it partially explains why an early signal can be predictive at all: whether a run eventually succeeds appears to be set substantially by properties of the specific initialization-and-warmup outcome, largely independent of which reward schedule follows. We looked for, but explicitly did not find, a usable signature of this in the four golden seeds’ `EFFECTIVE_RANK` values *before* any RL training began: three of four rank in the top quartile of all 48 seeds at that point, but the fourth ranks 36th of 48. With only 4 observations and a clear counterexample, we do not treat this as evidence and mention it only for transparency.

5.7 Cross-Task-Family Robustness: A Clean Null

TASK-B (two dependent tokens, chance success $\approx 0.045\%$) produced *zero* breakthroughs across all 48 runs, even with a proportionally larger step budget (6,138 vs. 4,092 steps, reflecting the $\sim 23\times$ rarer chance-level success), so every signal-discrimination statistic on TASK-B is undefined. We read this as a well-powered null on whether our pipeline transfers to a harder task family at this compute budget, not as evidence we simply did not try hard enough – but it leaves cross-task generalization genuinely untested, stated plainly as a limitation (§6.1) rather than a result in either direction.

5.8 What We Could Not Establish

We attempted a cross-architecture stretch check – LoRA fine-tuning [24] of a pretrained Qwen2.5-0.5B-Instruct model [25] under the same two-schedule GRPO comparison – as a first, partial test of whether our findings say anything beyond our toy architecture. It did not complete, failing on a dependency version conflict in the training environment (Appendix G) before producing any usable training signal. We report this as an attempted, incomplete check, not as a negative result: we have no evidence either for or against cross-architecture transfer of our findings.

6 Discussion

The question we set out to answer was whether supervised-grokking precursor signals transfer to RL-induced grokking. The honest answer, at the scale and sample size we could test, is a genuine but narrower “yes” than the question as originally posed suggests. `EFFECTIVE_RANK` predicts *how well* an RL training run eventually performs, robustly, independent of two separate replication checks, which is a real and useful finding. It does not reliably predict the *discrete* breakthrough event that motivated the original question – a practitioner monitoring `EFFECTIVE_RANK` during an RLVR run’s early plateau has real evidence about eventual model quality, but should not treat it as a reliable signal for whether a specific capability threshold will be crossed. Whether this reflects something structural about discrete phase transitions being harder to predict than continuous trajectories, or is simply an artifact of having only 7 (1, on the confirmatory batch alone) breakthrough events to work with, our data cannot resolve.

`POLICY_ENTROPY`’s exploratory result (§5.4) is, on its face, the largest effect we observe, and larger on the confirmatory batch than on the batch that would have selected it. We deliberately do not treat this as a second confirmed finding: by the same methodological standard we hold `EFFECTIVE_RANK` to, an effect that has not yet been tested on data collected after its selection is exploratory regardless of how clean it looks, and the honest next step is a fourth pre-registration round, not a retroactive promotion.

6.1 Limitations

We list these plainly rather than folding them into qualified language elsewhere. **Scale:** our policy is 821,504 parameters on synthetic modular arithmetic; RLVR in practice fine-tunes billion-parameter, pretrained models on natural-language reasoning, and we do not know how our findings extrapolate – this study is a first, careful, reproducible attempt at the underlying question under a real compute constraint, not an answer to how it scales. **Sample size for the discrete event:** 7 breakthroughs (1 on the confirmatory batch alone) is a small positive-class count by construction, since breakthroughs are rare even in the schedule designed to encourage them; §5.3’s null on the confirmatory batch is consistent with either a real absence of discrete-event predictability or insufficient power to detect it, and our data cannot distinguish these. **The entropy signal is not fully free:** our GRPO trainer includes a small entropy bonus needed to prevent a collapse spiral we diagnosed directly (§3.2.1), so `POLICY_ENTROPY`’s trajectory is partially shaped by explicit regularization, not an entirely emergent quantity. **Cross-task and cross-architecture generalization are open:** `TASK-B` is a clean null and the stretch check did not complete, so we have no evidence either way beyond `TASK-A`’s setting or our toy architecture.

7 Conclusion

We built a reproducible, compute-constrained pipeline to test whether early-warning signals for supervised-learning grokking transfer to RL-induced grokking, fixing a GRPO training-stability failure mode along the way that was a precondition for the question to be askable at all. Across three pre-registered rounds – one honestly-reported failure to replicate, and one signal that predicts eventual training reward robustly across an independent seed-batch replication and a reward-level confound check, but not the discrete breakthrough event that motivated the question – we arrive at a narrower, more precisely scoped finding than we set out to test, and report it as such, together with everything the pipeline could not establish.

Reproducibility Statement

Every number in this paper traces to a specific logged cell output, checkpoint file, or exported CSV/JSON artifact in the released pipeline; none are estimated or interpolated. Code, the pre-registration configuration for each round, all 144 runs’ per-step logs, and the scripts producing every table and figure are included in the supplementary material and will be released publicly. Seeds and environment are logged per run (Appendix F); §5.3 and Appendix B report every test we ran, including those that did not support our hypotheses.

References

- [1] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.
- [2] Ziming Liu, Ouail Kitouni, Niklas Nolte, Eric J Michaud, Max Tegmark, and Mike Williams. Towards understanding grokking: An effective theory of representation learning. *arXiv preprint arXiv:2205.10343*, 2022.
- [3] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations*, 2023.
- [4] Vikrant Varma, Rohin Shah, Zachary Kenton, János Kramár, and Ramana Kumar. Explaining grokking through circuit efficiency. *arXiv preprint arXiv:2309.02390*, 2023.
- [5] Vimal Thilak, Etai Littwin, Shuangfei Zhai, Omid Saremi, Roni Paiss, and Joshua Susskind. The slingshot mechanism: An empirical study of adaptive optimizers and the grokking phenomenon. *arXiv preprint arXiv:2206.04817*, 2022.
- [6] Tanishq Kumar, Blake Bordelon, Samuel J Gershman, and Cengiz Pehlevan. Grokking as the transition from lazy to rich training dynamics. In *International Conference on Learning Representations*, 2024.
- [7] Ziming Liu, Eric J Michaud, and Max Tegmark. Omnigrok: Grokking beyond algorithmic data. In *International Conference on Learning Representations*, 2023.
- [8] Yiyao Sun, Yuhao Cao, Pohao Huang, Haoyue Bai, Hannaneh Hajishirzi, Nouha Dziri, and Dawn Song. RL grokking recipe: How does RL unlock and transfer new algorithms in LLMs? In *International Conference on Learning Representations*, 2026.
- [9] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [10] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992.
- [11] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, et al. Understanding R1-Zero-Like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- [12] Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *15th European Signal Processing Conference (EUSIPCO)*, pages 606–610, 2007.
- [13] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.
- [14] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, et al. Deepseek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [16] Rui Wang, Zeming Wei, Guanzhang Yue, and Meng Sun. Revisiting hallucination detection with effective rank-based uncertainty. *arXiv preprint arXiv:2510.08389*, 2025.

-
- [17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
 - [18] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
 - [19] James A Hanley and Barbara J McNeil. The meaning and use of the area under a receiver operating characteristic (roc) curve. *Radiology*, 143(1):29–36, 1982.
 - [20] Charles Spearman. The proof and measurement of association between two things. *American Journal of Psychology*, 15(1):72–101, 1904.
 - [21] Carlo E Bonferroni. Teoria statistica delle classi e calcolo delle probabilità. *Pubblicazioni del R Istituto Superiore di Scienze Economiche e Commerciali di Firenze*, 8:3–62, 1936.
 - [22] Bradley Efron. Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
 - [23] Bradley Efron and Robert J Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994.
 - [24] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
 - [25] Qwen Team, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

A Full Hyperparameters

B Full Statistical Tables

C Lead-Time Analysis

For the 7 breakthrough runs, Table 8 reports the number of steps between each signal’s first crossing of a 2σ threshold (relative to its own first-quartile-of-training baseline) and the run’s eventual breakthrough step. All four signals show comparable median lead times of roughly 2,200–2,900 steps, though with only 7 runs these should be read as descriptive, not as a separately powered test.

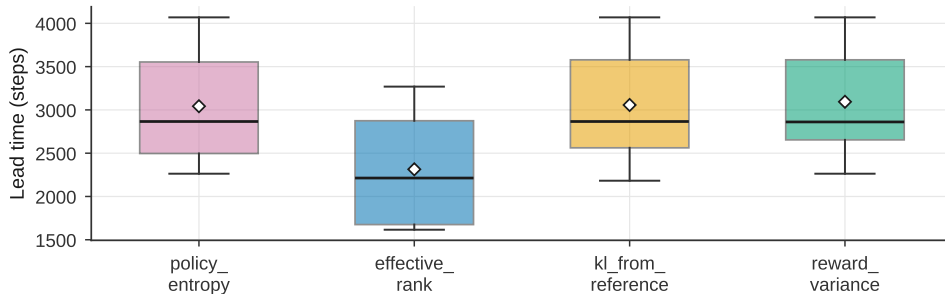


Figure 2: Distribution of lead time by signal, $n=7$ breakthrough runs.

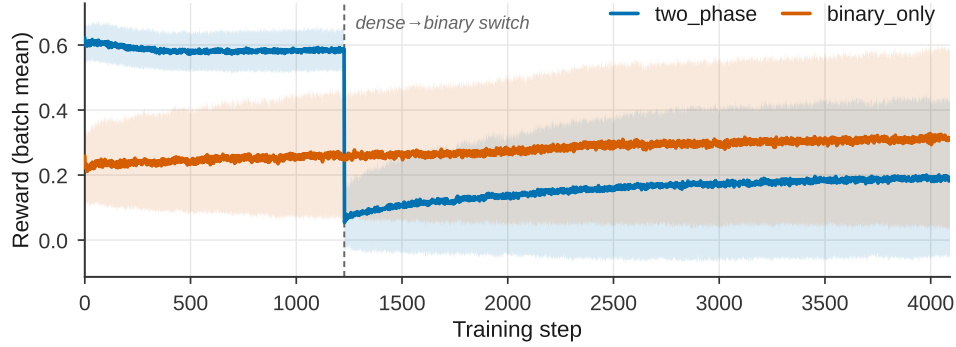


Figure 3: Reward trajectory (mean $\pm$ one standard deviation across seeds) for *two_phase* versus *binary_only* schedules on TASK-A, $n=48$ seeds per schedule. The dashed line marks the dense-to-binary reward switch at 30% of training for *two_phase*. Referenced from §5.

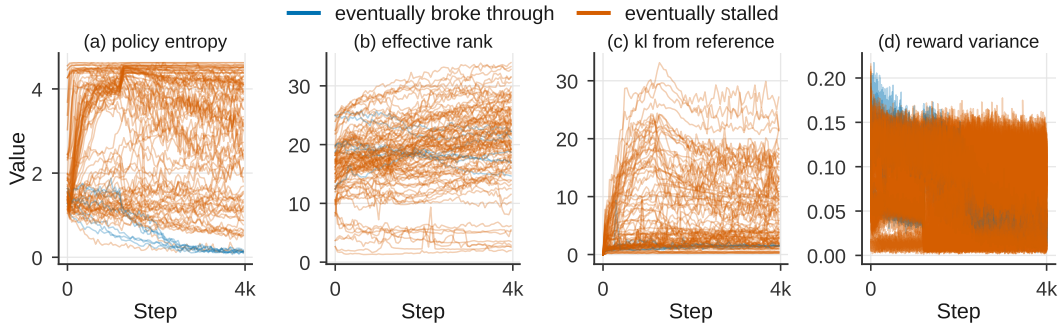


Figure 4: All four candidate signals’ full trajectories, all 96 TASK-A runs, colored by eventual outcome.

D Additional Figures

E Failure Analysis

At the primary (signal, window) pair, 29 of 96 runs (30.2%) are misclassified at the AUROC-optimal threshold (EFFECTIVE_RANK=18.598). Of these, 28 are false positives (early EFFECTIVE_RANK above threshold, run did not break through) and 1 is a false negative: seed 31 under *binary_only* broke through despite an early EFFECTIVE_RANK value (14.32) well below threshold. This is the same seed identified in §5.6 as the one “golden” seed whose pre-RL EFFECTIVE_RANK rank (36th of 48) does not match the other three golden seeds’ top-quartile values – the classifier’s one false negative and the one seed that did not fit the (explicitly rejected, §5.6) pre-RL signature pattern are the same run, which we note only as an internally consistent detail, not as an additional finding. Misclassifications split 12 *two_phase* / 17 *binary_only*, which we do not read as a strong schedule-specific pattern given the sample size.

F Compute and Environment

All runs used a single NVIDIA T4 GPU on Kaggle. Measured throughput at the final profiling checkpoint was 27,564.5 samples/sec (37.1 ms/step); the auto-calibration rates used for step-budget planning were 38.8 ms/step (TASK-A) and 48.0 ms/step (TASK-B), measured separately by a two-point timing protocol within a short calibration run to isolate marginal per-step cost from fixed one-time overhead (CUDA initialization, reference-policy cloning, probe construction). Peak GPU memory usage across the full pipeline was 1.27 GB. Total wall-clock time for the complete pipeline – both task families, all warmup runs, all 144 training runs, and all statistical analysis and figure

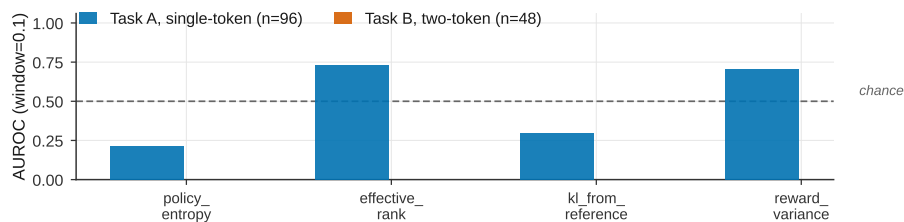


Figure 5: Discrimination AUROC by signal, TASK-A versus TASK-B at window 0.1. TASK-B bars are absent because every statistic is undefined (a single-outcome-class sample; §5.7).

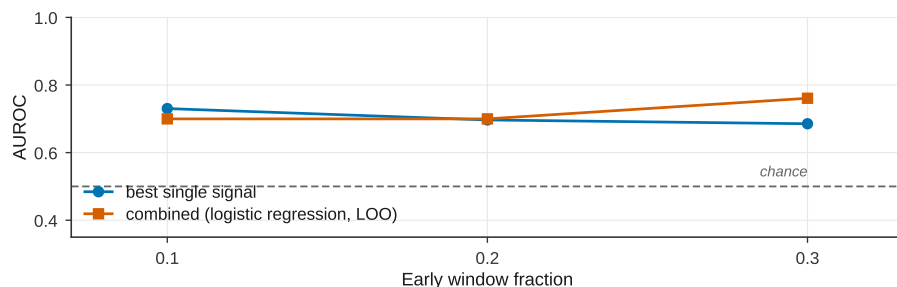


Figure 6: Best single signal versus the four-signal logistic-regression combination, by early-window fraction (§5.5).

generation – was 4.36 hours, comfortably inside the 11-hour session budget. Full package versions are exported alongside every run and included in the supplementary material.

F.1 Task Integrity Checks

Before any training, we verify empirically that each task’s reward function and vocabulary produce the expected random-policy success rate. TASK-A: empirical 1.043% versus theoretical 1.031% (relative error 1.17%). TASK-B: empirical 0.0390% versus theoretical 0.0448% (relative error 12.99%, expected to be noisier given how rare a double-correct random guess is). Both checks pass the pipeline’s validation threshold.

G Stretch Check: Attempted Cross-Architecture Test

We attempted to repeat a reduced version of the two-schedule comparison using LoRA fine-tuning [24] of Qwen2.5-0.5B-Instruct [25] (3 seeds, both schedules, 800 GRPO steps) as a first, partial check of whether our findings say anything about a real pretrained architecture. The attempt did not complete: dependency installation for this section pulled in a version of `torchao` (0.10.0) incompatible with the version required by the installed `transformers/peft` stack ($\geq 0.16.0$), raising an `ImportError` before any training step executed. This is an environment/dependency-resolution failure, not a result about the method itself, and we report it as an incomplete attempt rather than as a negative finding in either direction. We do not have evidence, positive or negative, about cross-architecture transfer of our results.

H Use of Large Language Models

An AI coding and writing assistant was used throughout this project: in designing and iteratively debugging the experimental pipeline (including identifying the GRPO normalization failure mode in §3.2.1 and the fixed-pair task redesign in §3.1), in building the statistical analysis code, and in drafting this manuscript from the resulting experimental logs and the authors’ direction. All experimental design decisions, the interpretation of results, and the decision to report non-replicating

findings plainly rather than omit them were made by the human author(s), who take full responsibility for the paper's content and claims.

Table 4: Complete hyperparameter configuration, both task families.

Parameter	Value
<i>Model</i>	
d_{model}	128
Layers	4
Attention heads	4
d_{ff}	512
Max sequence length	16
Total parameters	821,504
<i>Supervised warmup</i>	
Steps	3,000 (early-stopped at 60% train accuracy)
Batch size	128
Learning rate	3×10^{-3}
Weight decay	0.01
<i>GRPO</i>	
Prompts per step	64
Group size	16
Sampling temperature	0.9
Learning rate	2×10^{-3}
Weight decay	0.05
Gradient clip (global norm)	1.0
Entropy coefficient	0.05
Warmup fraction (two_phase dense phase)	0.3
Breakthrough threshold / window	0.9 success rate over 100 steps
<i>Task A (single-token)</i>	
Modulus	97
Total steps (calibrated)	4,092
Seeds	0–47 (48)
<i>Task B (two-token)</i>	
Moduli	97, 23
Total steps (calibrated)	6,138
Seeds	0–23 (24)
<i>Signal analysis</i>	
Early-window fractions tested	0.1, 0.2, 0.3
Bootstrap resamples	2,000

Table 5: Full discrimination-AUROC table, all signals, all windows, 96-run TASK-A sample. EFFECTIVE_RANK at $w=0.1$ (bold) is the pre-registered primary comparison and is evaluated at its own nominal α ; all other rows are secondary/exploratory and are shown with Bonferroni-corrected p -values across the 12-comparison family ($4 \text{ signals} \times 3 \text{ windows}$).

Signal	w	AUROC	95% CI	n_{break}	p	p_{Bonf}
EFFECTIVE_RANK	0.1	0.730	[0.508, 0.890]	7	0.0385	–
REWARD_VARIANCE	0.1	0.708	[0.446, 0.974]	7	0.0730	0.876
raw reward (baseline)	0.1	0.685	–	–	–	–
elapsed steps (baseline)	0.1	0.500	–	–	–	–
KL_FROM_REFERENCE	0.1	0.297	[0.165, 0.446]	7	0.0835	1.000
POLICY_ENTROPY	0.1	0.215	[0.095, 0.340]	7	0.0070	0.084
raw reward (baseline)	0.2	0.740	–	–	–	–
EFFECTIVE_RANK	0.2	0.697	[0.464, 0.866]	7	0.0795	0.954
REWARD_VARIANCE	0.2	0.648	[0.323, 0.992]	7	0.1910	1.000
elapsed steps (baseline)	0.2	0.500	–	–	–	–
KL_FROM_REFERENCE	0.2	0.270	[0.139, 0.412]	7	0.0465	0.558
POLICY_ENTROPY	0.2	0.177	[0.070, 0.301]	7	0.0010	0.012
raw reward (baseline)	0.3	0.868	–	–	–	–
EFFECTIVE_RANK	0.3	0.685	[0.467, 0.855]	7	0.0965	1.000
REWARD_VARIANCE	0.3	0.634	[0.296, 0.989]	7	0.2430	1.000
elapsed steps (baseline)	0.3	0.500	–	–	–	–
KL_FROM_REFERENCE	0.3	0.239	[0.117, 0.385]	7	0.0225	0.270
POLICY_ENTROPY	0.3	0.157	[0.050, 0.284]	7	0.0000	0.000

Table 6: Full continuous-outcome (Spearman r against final reward) table, all signals, all windows, $n=96$. Formatting follows Table 5.

Signal	w	r	95% CI	p	p_{Bonf}
EFFECTIVE_RANK	0.1	0.647	[0.489, 0.756]	<0.001	–
REWARD_VARIANCE	0.1	0.500	[0.313, 0.650]	<0.001	<0.001
KL_FROM_REFERENCE	0.1	–0.159	[–0.401, 0.071]	0.129	1.000
POLICY_ENTROPY	0.1	–0.817	[–0.869, –0.735]	<0.001	<0.001
EFFECTIVE_RANK	0.2	0.624	[0.465, 0.740]	<0.001	<0.001
REWARD_VARIANCE	0.2	0.446	[0.232, 0.615]	<0.001	<0.001
KL_FROM_REFERENCE	0.2	–0.164	[–0.401, 0.065]	0.119	1.000
POLICY_ENTROPY	0.2	–0.840	[–0.887, –0.767]	<0.001	<0.001
EFFECTIVE_RANK	0.3	0.622	[0.466, 0.737]	<0.001	<0.001
REWARD_VARIANCE	0.3	0.429	[0.219, 0.605]	<0.001	<0.001
KL_FROM_REFERENCE	0.3	–0.175	[–0.414, 0.056]	0.097	1.000
POLICY_ENTROPY	0.3	–0.853	[–0.896, –0.785]	<0.001	<0.001

Table 7: Partial correlation (controlling for raw reward at the same window), $w=0.1$, both metrics, all four signals. AUROC rows use the discrimination test on residualized signal values; r rows use the continuous-outcome test on the same residuals. p_{Bonf} corrects across the full 24-comparison family ($4 \text{ signals} \times 3 \text{ windows} \times 2 \text{ metrics}$).

Signal	Metric	Statistic	95% CI	p	p_{Bonf}
EFFECTIVE_RANK	AUROC	0.695	[0.506, 0.852]	0.0870	1.000
EFFECTIVE_RANK	r	0.630	[0.478, 0.751]	<0.001	<0.001
REWARD_VARIANCE	AUROC	0.737	[0.503, 0.969]	0.0360	0.864
REWARD_VARIANCE	r	0.528	[0.350, 0.670]	<0.001	<0.001
POLICY_ENTROPY	AUROC	0.258	[0.139, 0.386]	0.0310	0.744
POLICY_ENTROPY	r	-0.793	[-0.851, -0.707]	<0.001	<0.001
KL_FROM_REFERENCE	AUROC	0.112	[0.007, 0.311]	0.0000	0.000
KL_FROM_REFERENCE	r	-0.431	[-0.628, -0.208]	<0.001	<0.001

Table 8: Lead time (steps before breakthrough) by signal, $n=7$ breakthrough runs.

Signal	Mean	Median	Std.	n
EFFECTIVE_RANK	2314.0	2213.0	725.9	7
KL_FROM_REFERENCE	3056.9	2866.0	730.9	7
POLICY_ENTROPY	3042.6	2866.0	708.0	7
REWARD_VARIANCE	3094.0	2861.0	680.4	7